

Sincronizacao com o ZEHLA Brain

Guia completo de implementacao de codigo: pipeline de eventos, score engine, classificacao de leads e acoes automaticas em tempo real.

SMARTHOTEL SaaS • 2026 • v1.0

Pipeline de 5 estagios com BullMQ + Redis | 8 tipos de eventos com scoring automatico | Classificacao HOT/WARM/COLD | Deduplicacao via SHA-256 | Acoes automaticas por transicao de cluster

IMPLEMENTACAO COMPLETA

Sumario

1. Visao Geral da Arquitetura	3
1.1. Pipeline de 5 Estagios	3
2. Prisma Schema — Modelos de Dados	4
2.1. Modelo LeadEvent	4
2.2. Modelo Lead — Campos Expandidos	5
2.3. Modelo ActionLog	6
2.4. Modelo WebhookLog	6
3. Configuracao BullMQ + Redis	7
4. API Route — Captura de Eventos	8
5. Webhook Receiver — Integracoes Externas	9
6. Worker — Estagio Validate	11
7. Worker — Estagio Enrich	13
8. Worker — Estagio Classify (Score Engine)	16
9. Worker — Estagio Act (Action Dispatcher)	19
10. Capture Worker — Bridge para Validate	25
11. Orquestrador de Workers (Startup)	26
12. API de Dashboard — Metricas e Monitoramento	27
13. Client-Side Event Tracking	29
14. Configuracao e Deploy	32
14.1. Variaveis de Ambiente	32
14.2. PM2 Ecosystem Config	32
14.3. Docker Compose	33
14.4. Comandos de Deploy	34
15. Estrutura de Arquivos do Modulo	35
16. Fluxo Completo End-to-End	36

16.1. Exemplo Pratico	36
16.2. Log do Pipeline	36

1. Visao Geral da Arquitetura

A Fase 4 do ZEHLA Adaptive Sales Funnel implementa o cerebro cognitivo do sistema de vendas: a sincronizacao em tempo real com o ZEHLA Brain. Este modulo captura, valida, enriquece, classifica e age sobre cada interacao que um lead tem com qualquer canal da operacao. A arquitetura segue uma pipeline de 5 estagios processados via filas BullMQ com Redis, garantindo resiliencia, escalabilidade e idempotencia em toda a cadeia de processamento.

Cada evento de interacao (abertura de email, clique em link, visita a landing page, abertura do WhatsApp, visualizacao de anuncio, resposta no WhatsApp, inicio de trial ou pagamento realizado) e atribuido um peso numerico que alimenta um score acumulado por lead. Esse score determina automaticamente o cluster de classificacao: HOT (maior ou igual a 60 pontos), WARM (30 a 59 pontos) ou COLD (menor que 30 pontos). A transicao entre clusters dispara acoes automaticas como emails de nurturing, alertas para a equipe de vendas e ajustes de lances em campanhas publicitarias.

A infraestrutura tecnica utiliza Redis como message broker, BullMQ como sistema de filas, PostgreSQL via Prisma ORM como banco de dados persistente, e n8n/Make como orquestrador de webhooks para integracoes externas com Segment, mParticle, Sentry e outros servicos. O endpoint principal de captura de eventos e acessivel via POST /api/events/track, enquanto um webhook receiver em POST /api/events/webhook permite que plataformas externas enviem eventos de forma assincrona.

1.1. Pipeline de 5 Estagios

A pipeline de processamento de eventos segue uma sequencia estrita de 5 estagios, cada um com responsabilidade unica e bem definida. O fluxo comeca com a captura bruta do evento, seguida por validacao de integridade, enriquecimento com dados contextuais, classificacao por score e cluster, e finalmente a execucao de acoes automaticas baseadas na mudanca de estado do lead.

Estagio	Responsabilidade	Tecnologia
1. Capture	Receber e enfileirar eventos brutos	BullMQ + Redis
2. Validate	Verificar integridade e deduplicar	Prisma + Hash SHA-256
3. Enrich	Adicionar contexto (UTM, geo, device)	n8n/Make + Enrichment API
4. Classify	Recalcular score e atribuir cluster	Score Engine + Prisma
5. Act	Disparar acoes automaticas	Action Dispatcher + Webhooks

Tabela 1: Pipeline de 5 estagios do ZEHLA Brain

2. Prisma Schema — Modelos de Dados

O schema Prisma define os modelos centrais que sustentam toda a operacao do ZEHLE Brain. O modelo LeadEvent registra cada interacao com metadados, tipo, pontuacao e hash de deduplicacao. O modelo Lead e expandido com campos de score acumulado, cluster de classificacao, estagio do funil e data da ultima atividade. Alem disso, os modelos ActionLog e WebhookLog garantem auditoria completa de todas as acoes automaticas disparadas e integracoes externas recebidas.

2.1. Modelo LeadEvent

O modelo LeadEvent armazena cada evento de interacao do lead. O campo dedupHash combina email, fingerprint e tipo do evento em um hash SHA-256 para evitar processamento duplicado. O campo metadata permite armazenar dados adicionais como UTM parameters, device info e campanha de origem em formato JSON flexivel.

prisma/schema.prisma — LeadEvent

```
model LeadEvent {
  id          String   @id @default(cuid())
  leadId      String
  lead        Lead     @relation(fields: [leadId], references: [id])
  // Event identification
  eventType   String   // email_open, link_click, landing_visit,
                        // whatsapp_open, ad_view, whatsapp_reply,
                        // trial_started, payment_made
  eventSource String   // zehla_brain, webhook, api, segment
  sessionId   String?  // browser or app session ID
  dedupHash   String   @unique // SHA-256(email+fingerprint+type+timestamp_1h)
  // Scoring
  scoreImpact Int      // points added by this event
  // Contextual metadata
  metadata    Json?    // {utm_source, utm_campaign, device, ip_city, ...}
  // Processing status
  status      String   @default("pending")
  // pending, validated, enriched,
  // classified, acted, failed
  processedAt DateTime?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  @@index([leadId])
  @@index([eventType])
  @@index([dedupHash])
  @@index([status])
  @@map("lead_events")
}
```

2.2. Modelo Lead — Campos Expandidos

O modelo Lead recebe novos campos para suportar o sistema de scoring e classificacao. O campo score acumula a pontuacao total de todas as interacoes validadas. O campo cluster armazena a classificacao atual (HOT, WARM, COLD). O campo previousCluster registra o cluster anterior para detectar transicoes e disparar acoes apropriadas. O campo funnelStage rastreia o estagio atual do lead no funil de vendas adaptativo.

prisma/schema.prisma — Lead (expandido)

```
model Lead {
  id          String      @id @default(cuid())
  name        String
  email       String      @unique
  phone       String?
  company     String?
  pousadaName String?      // nome da pousada (hospedagem)
  city        String?
  state       String?
  latitude     Float?
  longitude    Float?
  // ZEHLA Brain scoring fields
  score       Int          @default(0)
  cluster     String       @default("COLD")
  // HOT, WARM, COLD
  previousCluster String?
  funnelStage String       @default("capture")
  // capture, classify, nurture, convert, retain
  // Lead Intelligence
  fingerprint String?      // browser fingerprint for dedup
  utmSource    String?
  utmCampaign  String?
  utmMedium    String?
  referrer     String?
  // Timestamps
  lastActivityAt DateTime?
  createdAt    DateTime    @default(now())
  updatedAt    DateTime    @updatedAt
  // Relations
  events       LeadEvent[]
  actionLogs   ActionLog[]
  @@index([email])
  @@index([cluster])
  @@index([score])
  @@index([funnelStage])
  @@map("leads")
}
```

2.3. Modelo ActionLog

O modelo ActionLog registra cada acao automatica disparada pelo pipeline. Isso inclui emails de nurturing enviados, alertas para a equipe de vendas, ajustes de lances em campanhas e notificacoes via WhatsApp. Cada log contem o tipo de acao, o lead afetado, o cluster que motivou a acao e os detalhes da execucao em formato JSON.

prisma/schema.prisma — ActionLog

```
model ActionLog {
  id          String   @id @default(cuid())
  leadId      String
  lead        Lead     @relation(fields: [leadId], references: [id])
  actionType  String   // nurture_email, sales_alert,
                      // bid_adjustment, whatsapp_notify
  trigger     String   // cluster_change, score_threshold,
                      // manual, scheduled
  cluster     String   // cluster do lead no momento da acao
  payload     Json?    // dados detalhados da acao executada
  status      String   @default("pending")
  // pending, sent, failed, retried
  error       String?
  retriedAt   DateTime?
  createdAt   DateTime @default(now())
  @@index([leadId])
  @@index([actionType])
  @@index([status])
  @@map("action_logs")
}
```

2.4. Modelo WebhookLog

O modelo WebhookLog registra todos os webhooks recebidos de plataformas externas como n8n, Make, Segment e mParticle. Isso garante rastreabilidade completa das integracoes, facilita debugging de falhas e permite reprocessamento manual quando necessario.

prisma/schema.prisma — WebhookLog

```
model WebhookLog {
  id          String   @id @default(cuid())
  source      String   // n8n, make, segment, mparticle
  eventType   String?
  payload     Json
  headers     Json?    // request headers for verification
  status      String   @default("received")
  // received, processed, failed
  response    String?
  createdAt   DateTime @default(now())
  @@index([source])
  @@index([status])
  @@map("webhook_logs")
}
```

3. Configuracao BullMQ + Redis

O sistema de filas BullMQ e o coracao da pipeline de processamento assincrono. Cada estagio da pipeline possui sua propria fila dedicada, permitindo processamento independente, retries automaticos em caso de falha e monitoramento em tempo real. A conexao com Redis e configurada via variaveis de ambiente para flexibilidade entre ambientes de desenvolvimento e producao. O modulo centraliza toda a logica de conexao e criacao de filas.

lib/redis.ts

```
// lib/redis.ts — Conexao Redis centralizada
import Redis from 'ioredis';
export const redis = new Redis(
  process.env.REDIS_URL || 'redis://localhost:6379',
  {
    maxRetriesPerRequest: null, // required for BullMQ
    enableReadyCheck: false,
    retryStrategy(times) {
      const delay = Math.min(times * 200, 5000);
      return delay;
    },
  },
);
redis.on('connect', () => {
  console.log('[Redis] Conectado com sucesso');
});
redis.on('error', (err) => {
  console.error('[Redis] Erro de conexao:', err.message);
});
```

lib/queues.ts

```
// lib/queues.ts — Definicao das filas BullMQ
import { Queue, Worker, Job } from 'bullmq';
import { redis } from './redis';
// Queue names para cada estagio da pipeline
export const QUEUE_NAMES = {
  CAPTURE: 'events:capture',
  VALIDATE: 'events:validate',
  ENRICH: 'events:enrich',
  CLASSIFY: 'events:classify',
  ACT: 'events:act',
} as const;
// Criacao das filas
export const captureQueue = new Queue(QUEUE_NAMES.CAPTURE, { connection: redis });
export const validateQueue = new Queue(QUEUE_NAMES.VALIDATE, { connection: redis });
export const enrichQueue = new Queue(QUEUE_NAMES.ENRICH, { connection: redis });
export const classifyQueue = new Queue(QUEUE_NAMES.CLASSIFY, { connection: redis });
export const actQueue = new Queue(QUEUE_NAMES.ACT, { connection: redis });
// Configuracao padrão para Workers
export const WORKER_CONFIG = {
  concurrency: 5,
  limiter: { max: 100, duration: 1000 }, // rate limit
  autorun: false, // iniciar manualmente em prod
} as const;
```


4. API Route — Captura de Eventos

O endpoint POST `/api/events/track` é a porta de entrada principal para captura de eventos. Ele recebe dados brutos de interação, valida o payload mínimo, adiciona o evento na fila de captura e retorna imediatamente um ID de rastreamento. O processamento real acontece de forma assíncrona via BullMQ, garantindo que a resposta HTTP seja rápida independente da complexidade do processamento. O endpoint suporta tanto requisições diretas do frontend quanto chamadas de serviços externos via API key.

`app/api/events/track/route.ts`

```
// app/api/events/track/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { captureQueue } from '@lib/queues';
import { v4 as uuidv4 } from 'uuid';
// Tipos de eventos validos com seus pesos
const EVENT_SCORES: Record<string, number> = {
  email_open: 5,
  link_click: 15,
  landing_visit: 10,
  whatsapp_open: 25,
  ad_view: 3,
  whatsapp_reply: 30,
  trial_started: 50,
  payment_made: 100,
};
const VALID_EVENTS = Object.keys(EVENT_SCORES);
export async function POST(req: NextRequest) {
  try {
    const body = await req.json();
    // Validacao basica do payload
    const { email, eventType, sessionId, fingerprint, metadata } = body;
    if (!email || !eventType) {
      return NextResponse.json(
        { error: 'email e eventType sao obrigatorios' },
        { status: 400 }
      );
    }
    if (!VALID_EVENTS.includes(eventType)) {
      return NextResponse.json(
        {
          error: `eventType invalido. Tipos validos: ${VALID_EVENTS.join(', ')}`
        },
        { status: 400 }
      );
    }
    // Gerar hash de deduplicacao
    const dedupWindow = Date.now() - (60 * 60 * 1000); // 1h window
    const dedupKey = `${email}:${fingerprint} || 'none':${eventType}:${Math.floor(dedupWindow / 3600000)}`;
    const crypto = await import('crypto');
    const dedupHash = crypto
      .createHash('sha256')
      .update(dedupKey)
```

```

        .digest('hex');
    // Criar job na fila de captura
    const job = await captureQueue.add('capture-event', {
        trackingId: uuidv4(),
        email: email.toLowerCase().trim(),
        eventType,
        eventSource: 'api',
        sessionId: sessionId || null,
        fingerprint: fingerprint || null,
        scoreImpact: EVENT_SCORES[eventType],
        dedupHash,
        metadata: metadata || {},
        receivedAt: new Date().toISOString(),
    }, {
        // Nao retriir eventos de captura (idempotente)
        attempts: 1,
        removeOnComplete: { count: 1000 },
        removeOnFail: { count: 500 },
    });
    return NextResponse.json({
        success: true,
        trackingId: job.data.trackingId,
        eventId: job.id,
        message: 'Evento enfileirado para processamento',
    }, { status: 202 });
} catch (error: any) {
    console.error('[Track] Erro:', error.message);
    return NextResponse.json(
        { error: 'Erro interno do servidor' },
        { status: 500 }
    );
}
}
}

```

5. Webhook Receiver — Integracoes Externas

O endpoint POST `/api/events/webhook` recebe eventos de plataformas externas como n8n, Make (Integromat), Segment e mParticle. Cada requisicao e logged no banco de dados para auditoria completa, o payload e normalizado para o formato interno do ZEHLA Brain, e o evento e roteado para a fila de captura para processamento assincrono. O endpoint suporta verificacao de assinatura HMAC para garantir autenticidade das requisicoes.

app/api/events/webhook/route.ts

```

// app/api/events/webhook/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { captureQueue } from '@lib/queues';
import { prisma } from '@lib/prisma';
import crypto from 'crypto';
// Mapeamento de eventos externos para internos
const EXTERNAL_EVENT_MAP: Record<string, string> = {
    // Segment events
    'Email Opened': 'email_open',

```

```

'Link Clicked':      'link_click',
'Page Viewed':      'landing_visit',
// n8n/Make custom events
'whatsapp_open':     'whatsapp_open',
'whatsapp_reply':    'whatsapp_reply',
'trial_started':     'trial_started',
'payment_made':      'payment_made',
// Facebook/Google Ads
'ad impression':     'ad_view',
'ad_click':          'ad_view',
};
const EVENT_SCORES: Record<string, number> = {
  email_open: 5, link_click: 15, landing_visit: 10,
  whatsapp_open: 25, ad_view: 3, whatsapp_reply: 30,
  trial_started: 50, payment_made: 100,
};
export async function POST(req: NextRequest) {
  try {
    const source = req.headers.get('x-webhook-source') || 'unknown';
    const signature = req.headers.get('x-webhook-signature') || '';
    const body = await req.json();
    // Log do webhook para auditoria
    await prisma.webhookLog.create({
      data: {
        source,
        eventType: body.event_type || body.event || body.type,
        payload: body,
        headers: Object.fromEntries(req.headers.entries()),
        status: 'received',
      },
    });
    // Verificar assinatura HMAC (se configurado)
    const webhookSecret = process.env.WEBHOOK_SECRET;
    if (webhookSecret && signature) {
      const expectedSig = crypto
        .createHmac('sha256', webhookSecret)
        .update(JSON.stringify(body))
        .digest('hex');
      if (signature !== expectedSig) {
        await prisma.webhookLog.updateMany({
          where: { source, status: 'received' },
          data: { status: 'failed', response: 'Invalid signature' },
        });
        return NextResponse.json(
          { error: 'Assinatura invalida' },
          { status: 401 }
        );
      }
    }
    // Normalizar evento
    const rawEventType = body.event_type || body.event || body.type || '';
    const internalEventType = EXTERNAL_EVENT_MAP[rawEventType]
      || EXTERNAL_EVENT_MAP[rawEventType.toLowerCase()]
      || null;
    if (!internalEventType) {
      return NextResponse.json(

```

```

        { error: `Tipo de evento nao mapeado: ${rawEventType}` },
        { status: 422 }
    );
}
const email = body.email || body.properties?.email
    || body.traits?.email || body.userId;
if (!email) {
    return NextResponse.json(
        { error: 'email nao encontrado no payload' },
        { status: 422 }
    );
}
// Deduplicacao
const dedupKey = `${email}:${rawEventType}:${Math.floor(Date.now() / 3600000)}`;
const dedupHash = crypto.createHash('sha256').update(dedupKey).digest('hex');
// Enfileirar para processamento
await captureQueue.add('webhook-event', {
    email: email.toLowerCase().trim(),
    eventType: internalEventType,
    eventSource: source,
    sessionId: body.sessionId || null,
    fingerprint: body.fingerprint || null,
    scoreImpact: EVENT_SCORES[internalEventType],
    dedupHash,
    metadata: {
        originalEvent: rawEventType,
        externalId: body.id || body.messageId,
        source: source,
        campaign: body.campaign || body.properties?.campaign,
        ...body.properties,
        ...body.traits,
    },
    receivedAt: new Date().toISOString(),
});
return NextResponse.json({
    success: true,
    message: 'Webhook processado e evento enfileirado',
    mappedEvent: internalEventType,
});
} catch (error: any) {
    console.error('[Webhook] Erro:', error.message);
    return NextResponse.json(
        { error: 'Erro interno do servidor' },
        { status: 500 }
    );
}
}

```

6. Worker — Estagio Validate

O worker de validacao e responsavel por verificar a integridade do evento, realizar deduplicacao via hash SHA-256 e garantir a existencia do lead no banco de dados. Se o lead nao existe, o worker cria um novo registro com os dados disponiveis. A deduplicacao impede que o mesmo evento seja processado multiplas

vezes dentro de uma janela de 1 hora, mesmo que recebido por canais diferentes. Eventos validados com sucesso são encaminhados para a fila de enriquecimento.

workers/validateWorker.ts

```
// workers/validateWorker.ts
import { Worker, Job } from 'bullmq';
import { redis } from '@lib/redis';
import { QUEUE_NAMES, WORKER_CONFIG } from '@lib/queues';
import { prisma } from '@lib/prisma';
import { enrichQueue } from '@lib/queues';

const validateWorker = new Worker(
  QUEUE_NAMES.VALIDATE,
  async (job: Job) => {
    const data = job.data;
    console.log(`[Validate] Processando evento: ${data.eventType} para ${data.email}`);
    // 1. Verificar deduplicacao
    const existingEvent = await prisma.leadEvent.findUnique({
      where: { dedupHash: data.dedupHash },
    });
    if (existingEvent) {
      console.log(`[Validate] Evento duplicado ignorado: ${data.dedupHash.slice(0, 12)}`);
      return { status: 'duplicate', eventId: existingEvent.id };
    }
    // 2. Buscar ou criar lead
    let lead = await prisma.lead.findUnique({
      where: { email: data.email },
    });
    if (!lead) {
      lead = await prisma.lead.create({
        data: {
          email: data.email,
          name: data.metadata?.name || data.metadata?.first_name
            ? `${data.metadata?.first_name} ${data.metadata?.last_name || ''}`
            : `Lead ${data.email.split('@')[0]}`,
          phone: data.metadata?.phone || null,
          pousadaName: data.metadata?.pousada_name || null,
          city: data.metadata?.city || null,
          state: data.metadata?.state || null,
          fingerprint: data.fingerprint,
          utmSource: data.metadata?.utm_source || null,
          utmCampaign: data.metadata?.utm_campaign || null,
          utmMedium: data.metadata?.utm_medium || null,
          lastActivityAt: new Date(),
        },
      });
    }
    console.log(`[Validate] Novo lead criado: ${lead.id}`);
    // 3. Criar registro do evento
    const event = await prisma.leadEvent.create({
      data: {
        leadId: lead.id,
        eventType: data.eventType,
        eventSource: data.eventSource,
        sessionId: data.sessionId,
        dedupHash: data.dedupHash,
      },
    });
    enrichQueue.add(job.id, { email: data.email });
  },
  {
    redis,
    config: WORKER_CONFIG,
  }
);
```

```

        scoreImpact: data.scoreImpact,
        metadata: data.metadata || {},
        status: 'validated',
        processedAt: new Date(),
      },
    });
    // 4. Atualizar ultima atividade do lead
    await prisma.lead.update({
      where: { id: lead.id },
      data: { lastActivityAt: new Date() },
    });
    // 5. Encaminhar para enriquecimento
    await enrichQueue.add('enrich-event', {
      eventId: event.id,
      leadId: lead.id,
      email: data.email,
      eventType: data.eventType,
      eventSource: data.eventSource,
      metadata: data.metadata || {},
      trackingId: data.trackingId,
    });
    return { status: 'validated', eventId: event.id, leadId: lead.id };
  },
  {
    connection: redis,
    concurrency: WORKER_CONFIG.concurrency,
    limiter: WORKER_CONFIG.limiter,
  }
);
validateWorker.on('completed', (job) => {
  console.log(`[Validate] Job ${job.id} completado`);
});
validateWorker.on('failed', (job, err) => {
  console.error(`[Validate] Job ${job?.id} falhou:`, err.message);
});
export default validateWorker;

```

7. Worker — Estagio Enrich

O worker de enriquecimento adiciona dados contextuais ao evento antes da classificacao. Ele busca informacoes adicionais como geolocalizacao por IP, dados do dispositivo, informacoes da campanha de marketing e historico de interacoes recentes do lead. O enriquecimento utiliza APIs externas como IPGeolocation e Clearbit,lem de consultar o banco de dados interno para construir um perfil completo do lead. O resultado e encaminhado para o estagio de classificacao com todos os dados consolidados.

workers/enrichWorker.ts

```

// workers/enrichWorker.ts
import { Worker, Job } from 'bullmq';
import { redis } from '@lib/redis';
import { QUEUE_NAMES, WORKER_CONFIG } from '@lib/queues';
import { prisma } from '@lib/prisma';
import { classifyQueue } from '@lib/queues';

```

```

// Tipos de enriquecimento disponiveis
type EnrichmentType = 'geo' | 'device' | 'campaign' | 'history';
async function enrichGeo(ip: string): Promise<any> {
  try {
    const response = await fetch(
      `https://ipapi.co/${ip}/json/`,
      { next: { revalidate: 86400 } }
    );
    if (!response.ok) return null;
    const data = await response.json();
    return {
      city: data.city,
      region: data.region,
      country: data.country_name,
      latitude: data.latitude,
      longitude: data.longitude,
      timezone: data.timezone,
    };
  } catch {
    return null;
  }
}

async function enrichLeadHistory(leadId: string): Promise<any> {
  // Buscar eventos dos ultimos 30 dias
  const thirtyDaysAgo = new Date();
  thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);
  const recentEvents = await prisma.leadEvent.findMany({
    where: {
      leadId,
      createdAt: { gte: thirtyDaysAgo },
    },
    orderBy: { createdAt: 'desc' },
    take: 50,
    select: { eventType: true, createdAt: true, scoreImpact: true },
  });
  const eventCounts: Record<string, number> = {};
  let totalScore30d = 0;
  for (const evt of recentEvents) {
    eventCounts[evt.eventType] = (eventCounts[evt.eventType] || 0) + 1;
    totalScore30d += evt.scoreImpact;
  }
  return {
    eventsLast30Days: recentEvents.length,
    eventCounts,
    totalScore30d,
    mostFrequentEvent: Object.entries(eventCounts)
      .sort(([, a], [, b]) => b - a)[0]?.[0] || null,
  };
}

const enrichWorker = new Worker(
  QUEUE_NAMES.ENRICH,
  async (job: Job) => {
    const { eventId, leadId, metadata } = job.data;
    console.log(`[Enrich] Enriquecendo evento: ${eventId}`);
    const enrichedData: any = { ...metadata };
    // Enriquecimento paralelo

```

```

const [geoData, historyData] = await Promise.all([
  metadata?.ip ? enrichGeo(metadata.ip) : Promise.resolve(null),
  enrichLeadHistory(leadId),
]);
if (geoData) {
  enrichedData.geo = geoData;
  // Atualizar dados de localizacao do lead
  if (geoData.city || geoData.latitude) {
    await prisma.lead.update({
      where: { id: leadId },
      data: {
        city: geoData.city || undefined,
        state: geoData.region || undefined,
        latitude: geoData.latitude || undefined,
        longitude: geoData.longitude || undefined,
      },
    });
  }
}
if (historyData) {
  enrichedData.history = historyData;
}
// Extrair info de campanha se presente
if (metadata?.utm_source) {
  enrichedData.campaign = {
    source: metadata.utm_source,
    medium: metadata.utm_medium,
    campaign: metadata.utm_campaign,
    content: metadata.utm_content,
    term: metadata.utm_term,
  };
}
// Extrair info de device se presente
if (metadata?.user_agent) {
  enrichedData.device = {
    userAgent: metadata.user_agent,
    browser: parseBrowser(metadata.user_agent),
    os: parseOS(metadata.user_agent),
    mobile: /Mobile|Android|iPhone/i.test(metadata.user_agent),
  };
}
// Atualizar metadados do evento
await prisma.leadEvent.update({
  where: { id: eventId },
  data: {
    metadata: enrichedData,
    status: 'enriched',
  },
});
// Encaminhar para classificacao
await classifyQueue.add('classify-event', {
  eventId,
  leadId,
  eventType: job.data.eventType,
  enrichedMetadata: enrichedData,
  trackingId: job.data.trackingId,
});

```



```

});
return { status: 'enriched', eventId, enrichmentsApplied: Object.keys(enrichedData).length };
},
{
  connection: redis,
  concurrency: WORKER_CONFIG.concurrency,
  limiter: { max: 50, duration: 1000 },
}
);
// Helpers simples de parser
function parseBrowser(ua: string): string {
  if (ua.includes('Chrome') && !ua.includes('Edg')) return 'Chrome';
  if (ua.includes('Firefox')) return 'Firefox';
  if (ua.includes('Safari') && !ua.includes('Chrome')) return 'Safari';
  if (ua.includes('Edg')) return 'Edge';
  return 'Unknown';
}
function parseOS(ua: string): string {
  if (ua.includes('Windows')) return 'Windows';
  if (ua.includes('Mac OS')) return 'macOS';
  if (ua.includes('Android')) return 'Android';
  if (ua.includes('iPhone') || ua.includes('iPad')) return 'iOS';
  if (ua.includes('Linux')) return 'Linux';
  return 'Unknown';
}
enrichWorker.on('failed', (job, err) => {
  console.error(`[Enrich] Job ${job?.id} falhou:`, err.message);
});
export default enrichWorker;

```

8. Worker — Estagio Classify (Score Engine)

O worker de classificacao e o nucleo do ZEHLA Brain. Ele recalcula o score total do lead, determina o cluster (HOT, WARM, COLD), detecta transicoes de cluster e encaminha para o estagio de acao quando necessario. O score e calculado somando todos os impactos de eventos validados do lead. A classificacao utiliza thresholds predefinidos: leads com score maior ou igual a 60 sao HOT (prontos para conversao), entre 30 e 59 sao WARM (requerem nurturing), e abaixo de 30 sao COLD (topo do funil).

Regra de Classificacao: score $\geq$ 60 = HOT | 30 $\leq$ score < 60 = WARM | score < 30 = COLD

Tipo de Evento	Score	Descricao
email_open	+5	Lead abriu email de campanha
link_click	+15	Lead clicou em link do email
landing_visit	+10	Lead visitou pagina de destino
whatsapp_open	+25	Lead abriu conversa no WhatsApp
ad_view	+3	Lead visualizou anuncio

whatsapp_reply	+30	Lead respondeu no WhatsApp
trial_started	+50	Lead iniciou periodo trial
payment_made	+100	Lead realizou pagamento

Tabela 2: Tipos de eventos e impactos no score

workers/classifyWorker.ts

```
// workers/classifyWorker.ts
import { Worker, Job } from 'bullmq';
import { redis } from '@lib/redis';
import { QUEUE_NAMES, WORKER_CONFIG } from '@lib/queues';
import { prisma } from '@lib/prisma';
import { actQueue } from '@lib/queues';
// Thresholds de classificacao
const CLUSTER_THRESHOLDS = {
  HOT: 60,
  WARM: 30,
  COLD: 0,
} as const;
type Cluster = 'HOT' | 'WARM' | 'COLD';
function determineCluster(score: number): Cluster {
  if (score >= CLUSTER_THRESHOLDS.HOT) return 'HOT';
  if (score >= CLUSTER_THRESHOLDS.WARM) return 'WARM';
  return 'COLD';
}
const classifyWorker = new Worker(
  QUEUE_NAMES.CLASSIFY,
  async (job: Job) => {
    const { eventId, leadId, eventType } = job.data;
    console.log(`[Classify] Classificando lead: ${leadId}`);
    // 1. Buscar lead atual
    const lead = await prisma.lead.findUnique({
      where: { id: leadId },
      select: {
        id: true, score: true, cluster: true,
        funnelStage: true, email: true, name: true,
      },
    });
    if (!lead) throw new Error(`Lead nao encontrado: ${leadId}`);
    // 2. Buscar todos os eventos validados do lead
    const allEvents = await prisma.leadEvent.findMany({
      where: {
        leadId,
        status: { in: ['validated', 'enriched', 'classified'] },
      },
      select: { scoreImpact: true },
    });
    // 3. Recalcular score total
    const newScore = allEvents.reduce((sum, evt) => sum + evt.scoreImpact, 0);
    // 4. Determinar novo cluster
    const previousCluster = lead.cluster as Cluster;
    const newCluster = determineCluster(newScore);
    // 5. Atualizar lead
    await prisma.lead.update({
```

```

    where: { id: leadId },
    data: {
      score: newScore,
      previousCluster: newCluster !== previousCluster ? previousCluster : null,
      cluster: newCluster,
      funnelStage: determineFunnelStage(newCluster),
      lastActivityAt: new Date(),
    },
  });
// 6. Atualizar status do evento
await prisma.leadEvent.update({
  where: { id: eventId },
  data: { status: 'classified' },
});
// 7. Verificar transicao de cluster
const clusterChanged = newCluster !== previousCluster;
const result = {
  leadId,
  previousScore: lead.score,
  newScore,
  previousCluster,
  newCluster,
  clusterChanged,
  funnelStage: determineFunnelStage(newCluster),
};
console.log(
  `[Classify] Lead ${lead.email}: ${lead.score} (${previousCluster}) ` +
  `-> ${newScore} (${newCluster})${clusterChanged ? ' *** TRANSICAO ***' : ''}`
);
// 8. Se houve transicao, encaminhar para acao
if (clusterChanged) {
  await actQueue.add('cluster-transition', {
    leadId,
    leadEmail: lead.email,
    leadName: lead.name,
    previousCluster,
    newCluster,
    previousScore: lead.score,
    newScore,
    eventType,
    enrichedMetadata: job.data.enrichedMetadata,
    trackingId: job.data.trackingId,
  }, {
    priority: newCluster === 'HOT' ? 1 : 5, // alta prioridade para HOT
  });
}
return result;
},
{
  connection: redis,
  concurrency: WORKER_CONFIG.concurrency,
}
);
function determineFunnelStage(cluster: Cluster): string {
  switch (cluster) {
    case 'HOT': return 'convert';
  }
}

```

```

    case 'WARM': return 'nurture';
    case 'COLD': return 'capture';
    default:     return 'capture';
  }
}
classifyWorker.on('failed', (job, err) => {
  console.error(`[Classify] Job ${job?.id} falhou:`, err.message);
});
export default classifyWorker;

```

9. Worker — Estagio Act (Action Dispatcher)

O worker de acoes e o executor final da pipeline. Quando uma transicao de cluster e detectada, este worker dispara as acoes automaticas apropriadas para o novo estado do lead. Para transicoes para HOT, dispara alertas urgentes para a equipe de vendas via WhatsApp e email, alem de ajustar lances de campanhas publicitarias. Para transicoes para WARM, envia sequencias de email de nurturing e programas follow-ups. Para transicoes para COLD, pode reduzir investimentos ou reativar campanhas de reconquista dependendo do contexto.

workers/actWorker.ts

```

// workers/actWorker.ts
import { Worker, Job } from 'bullmq';
import { redis } from '@lib/redis';
import { QUEUE_NAMES, WORKER_CONFIG } from '@lib/queues';
import { prisma } from '@lib/prisma';
interface ActionPayload {
  leadId: string;
  leadEmail: string;
  leadName: string;
  previousCluster: string;
  newCluster: string;
  previousScore: number;
  newScore: number;
  eventType: string;
  enrichedMetadata?: any;
  trackingId?: string;
}
// Definicao de acoes por transicao de cluster
const CLUSTER_ACTIONS: Record<string, string[]> = {
  'COLD->WARM': [
    'send_nurture_email_welcome',
    'activate_remarketing_campaign',
  ],
  'COLD->HOT': [
    'send_sales_alert_urgent',
    'activate_whatsapp_sequence',
    'adjust_ad_bid_increase',
    'send_trial_invitation',
  ],
  'WARM->HOT': [
    'send_sales_alert_hot_lead',
    'send_conversion_email',
  ],
}

```

```

    'activate_whatsapp_sequence',
    'adjust_ad_bid_increase',
  ],
  'WARM->COLD': [
    'send_reactivation_email',
    'reduce_ad_spend',
  ],
  'HOT->WARM': [
    'send_nurture_email',
    'schedule_follow_up',
    'adjust_ad_bid_normal',
  ],
  'HOT->COLD': [
    'send_churn_prevention_email',
    'pause_aggressive_campaigns',
    'create_winback_task',
  ],
];
};
async function executeAction(
  actionType: string,
  payload: ActionPayload
): Promise<{ success: boolean; error?: string; details?: any }> {
  try {
    switch (actionType) {
      case 'send_sales_alert_urgent':
      case 'send_sales_alert_hot_lead':
        return await sendSalesAlert(payload, actionType === 'send_sales_alert_urgent');
      case 'send_nurture_email':
      case 'send_nurture_email_welcome':
      case 'send_conversion_email':
      case 'send_reactivation_email':
      case 'send_churn_prevention_email':
        return await sendEmail(payload, actionType);
      case 'activate_whatsapp_sequence':
        return await activateWhatsApp(payload);
      case 'adjust_ad_bid_increase':
      case 'adjust_ad_bid_normal':
        return await adjustAdBid(payload, actionType.includes('increase'));
      case 'activate_remarketing_campaign':
        return await activateRemarketing(payload);
      case 'send_trial_invitation':
        return await sendTrialInvitation(payload);
      case 'schedule_follow_up':
        return await scheduleFollowUp(payload);
      case 'reduce_ad_spend':
      case 'pause_aggressive_campaigns':
        return await adjustCampaignSpend(payload, actionType === 'pause_aggressive_campaigns');
      case 'create_winback_task':
        return await createWinbackTask(payload);
      default:
        console.warn(`[Act] Acao nao implementada: ${actionType}`);
        return { success: false, error: 'Acao nao implementada' };
    }
  } catch (error: any) {
    return { success: false, error: error.message };
  }
}

```

```

}
// === Implementacoes de acoes ===
async function sendSalesAlert(payload: ActionPayload, urgent: boolean) {
  const message = urgent
    ? `URGENTE: Lead HOT detectado!\n\nNome: ${payload.leadName}\n`
    + `Email: ${payload.leadEmail}\nScore: ${payload.newScore}\n`
    + `Evento: ${payload.eventType}\n\nAcao imediata recomendada!`
    : `Lead ficou HOT:\n\nNome: ${payload.leadName}\n`
    + `Email: ${payload.leadEmail}\nScore: ${payload.newScore}`;
  // Enviar via API do WhatsApp Business
  const webhookUrl = process.env.ZEHLA_WHATSAPP_WEBHOOK_URL;
  if (webhookUrl) {
    await fetch(webhookUrl, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        type: 'sales_alert',
        urgent,
        phone: process.env.SALES_TEAM_PHONE,
        message,
        leadData: {
          id: payload.leadId,
          name: payload.leadName,
          email: payload.leadEmail,
          score: payload.newScore,
        },
      }),
    });
  }
  // Log da acao
  await prisma.actionLog.create({
    data: {
      leadId: payload.leadId,
      actionType: urgent ? 'sales_alert_urgent' : 'sales_alert',
      trigger: 'cluster_change',
      cluster: payload.newCluster,
      payload: { message, urgent },
      status: 'sent',
    },
  });
  return { success: true, details: { channel: 'whatsapp', urgent } };
}

async function sendEmail(payload: ActionPayload, templateId: string) {
  // Integracao com Delivery Machine (email marketing)
  const emailApiUrl = process.env.EMAIL_API_URL;
  if (!emailApiUrl) {
    console.warn('[Act] EMAIL_API_URL nao configurada');
    return { success: false, error: 'Email API not configured' };
  }

  const templateMap: Record<string, string> = {
    send_nurture_email: 'nurture_sequence_1',
    send_nurture_email_welcome: 'welcome_nurture',
    send_conversion_email: 'conversion_offer',
    send_reactivation_email: 'reactivation_campaign',
    send_churn_prevention_email: 'churn_prevention',
  };
};

```

```

const response = await fetch(emailApiUrl, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${process.env.EMAIL_API_KEY}` },
  body: JSON.stringify({
    templateId: templateMap[templateId] || templateId,
    to: payload.leadEmail,
    variables: {
      name: payload.leadName,
      score: payload.newScore,
      cluster: payload.newCluster,
      pousadaName: payload.enrichedMetadata?.pousada_name || null,
    },
  }),
});
await prisma.actionLog.create({
  data: {
    leadId: payload.leadId,
    actionType: templateId,
    trigger: 'cluster_change',
    cluster: payload.newCluster,
    payload: { templateId, email: payload.leadEmail },
    status: response.ok ? 'sent' : 'failed',
  },
});
return { success: response.ok, details: { template: templateId, status: response.status } };
}

async function activateWhatsApp(payload: ActionPayload) {
  // Disparar sequencia de mensagens via WhatsApp Business API
  console.log(`[Act] Ativando sequencia WhatsApp para ${payload.leadEmail}`);
  await prisma.actionLog.create({
    data: {
      leadId: payload.leadId,
      actionType: 'whatsapp_sequence',
      trigger: 'cluster_change',
      cluster: payload.newCluster,
      status: 'sent',
      payload: { email: payload.leadEmail, phone: payload.enrichedMetadata?.phone },
    },
  });
  return { success: true, details: { channel: 'whatsapp_sequence' } };
}

async function adjustAdBid(payload: ActionPayload, increase: boolean) {
  // Ajustar lances de campanhas (Meta Ads / Google Ads)
  const multiplier = increase ? 1.5 : 1.0;
  console.log(`[Act] Ajustando bid ${increase ? '+50%' : 'normal'} para lead ${payload.leadEmail}`);
  await prisma.actionLog.create({
    data: {
      leadId: payload.leadId,
      actionType: 'bid_adjustment',
      trigger: 'cluster_change',
      cluster: payload.newCluster,
      status: 'sent',
      payload: { multiplier, increase },
    },
  });
  return { success: true, details: { multiplier } };
}

```

```

}
async function activateRemarketing(payload: ActionPayload) {
  await prisma.actionLog.create({
    data: {
      leadId: payload.leadId,
      actionType: 'remarketing_activated',
      trigger: 'cluster_change',
      cluster: payload.newCluster,
      status: 'sent',
    },
  });
  return { success: true };
}

async function sendTrialInvitation(payload: ActionPayload) {
  await prisma.actionLog.create({
    data: {
      leadId: payload.leadId,
      actionType: 'trial_invitation',
      trigger: 'cluster_change',
      cluster: payload.newCluster,
      status: 'sent',
      payload: { email: payload.leadEmail },
    },
  });
  return { success: true };
}

async function scheduleFollowUp(payload: ActionPayload) {
  // Agendar follow-up via BullMQ delayed job
  const { captureQueue } = await import('@lib/queues');
  await captureQueue.add('scheduled-followup', {
    leadId: payload.leadId,
    email: payload.leadEmail,
    type: 'follow_up_reminder',
  }, { delay: 24 * 60 * 60 * 1000 }); // 24h
  return { success: true, details: { scheduledIn: '24h' } };
}

async function adjustCampaignSpend(payload: ActionPayload, pause: boolean) {
  await prisma.actionLog.create({
    data: {
      leadId: payload.leadId,
      actionType: pause ? 'campaigns_paused' : 'spend_reduced',
      trigger: 'cluster_change',
      cluster: payload.newCluster,
      status: 'sent',
    },
  });
  return { success: true };
}

async function createWinbackTask(payload: ActionPayload) {
  await prisma.actionLog.create({
    data: {
      leadId: payload.leadId,
      actionType: 'winback_task_created',
      trigger: 'cluster_change',
      cluster: payload.newCluster,
      status: 'sent',
    },
  });
  return { success: true };
}

```



```

    },
  });
  return { success: true };
}
// === Worker Principal ===
const actWorker = new Worker(
  QUEUE_NAMES.ACT,
  async (job: Job) => {
    const payload = job.data as ActionPayload;
    const transitionKey = `${payload.previousCluster}->${payload.newCluster}`;
    console.log(
      `[Act] Processando transicao: ${transitionKey} para lead ${payload.leadEmail}`
    );
    const actions = CLUSTER_ACTIONS[transitionKey] || [];
    if (actions.length === 0) {
      console.log(`[Act] Nenhuma acao definida para transicao: ${transitionKey}`);
      return { status: 'no_actions', transitionKey };
    }
    const results = await Promise.allSettled(
      actions.map(async (actionType) => {
        const result = await executeAction(actionType, payload);
        console.log(
          `[Act] ${actionType}: ${result.success ? 'OK' : 'FAIL'} ` +
          `${result.error || ''}`
        );
        return { actionType, ...result };
      })
    );
    const succeeded = results.filter(r => r.status === 'fulfilled').length;
    const failed = results.filter(r => r.status === 'rejected').length;
    console.log(
      `[Act] Transicao ${transitionKey}: ${succeeded} OK, ${failed} FAIL`
    );
    return {
      status: 'completed',
      transition: transitionKey,
      actionsExecuted: actions.length,
      succeeded,
      failed,
    };
  },
  {
    connection: redis,
    concurrency: 3,
  }
);
actWorker.on('failed', (job, err) => {
  console.error(`[Act] Job ${job?.id} falhou:`, err.message);
});
export default actWorker;

```

10. Capture Worker — Bridge para Validate

O Capture Worker é o primeiro worker da pipeline e atua como uma ponte entre o recebimento do evento e o estágio de validação. Embora os eventos já sejam adicionados diretamente a fila de captura pelo endpoint API, o Capture Worker processa essa fila e encaminha cada evento para a fila de validação. Isso permite adicionarmos lógica pre-validação como rate limiting por IP, throttling por lead e priorização de eventos de alto valor como `payment_made` e `trial_started`.

workers/captureWorker.ts

```
// workers/captureWorker.ts
import { Worker, Job } from 'bullmq';
import { redis } from '@lib/redis';
import { QUEUE_NAMES, WORKER_CONFIG } from '@lib/queues';
import { validateQueue } from '@lib/queues';
// Prioridade por tipo de evento (menor = maior prioridade)
const EVENT_PRIORITY: Record<string, number> = {
  payment_made: 1,
  trial_started: 2,
  whatsapp_reply: 3,
  whatsapp_open: 4,
  link_click: 5,
  landing_visit: 6,
  email_open: 7,
  ad_view: 8,
};

const captureWorker = new Worker(
  QUEUE_NAMES.CAPTURE,
  async (job: Job) => {
    const data = job.data;
    console.log(`[Capture] Evento recebido: ${data.eventType} (${data.email})`);
    // Rate limiting: max 50 eventos por lead por hora
    const rateLimitKey = `ratelimit:${data.email}:${Math.floor(Date.now() / 3600000)}`;
    const currentCount = await redis.incr(rateLimitKey);
    if (currentCount === 1) {
      await redis.expire(rateLimitKey, 3600);
    }
    if (currentCount > 50) {
      console.warn(`[Capture] Rate limit atingido para ${data.email}`);
      return { status: 'rate_limited', count: currentCount };
    }
    // Encaminhar para validacao com prioridade
    const priority = EVENT_PRIORITY[data.eventType] || 5;
    await validateQueue.add('validate-event', data, {
      priority,
      attempts: 3,
      backoff: { type: 'exponential', delay: 2000 },
    });
    return { status: 'forwarded_to_validate', priority };
  },
  {
    connection: redis,
    concurrency: 10,
    limiter: { max: 200, duration: 1000 },
  }
);
```

```

    }
  );
  captureWorker.on('completed', (job) => {
    console.log(`[Capture] Job ${job.id} processado`);
  });
  captureWorker.on('failed', (job, err) => {
    console.error(`[Capture] Job ${job?.id} falhou:`, err.message);
  });
  export default captureWorker;

```

11. Orquestrador de Workers (Startup)

O orquestrador centraliza a inicializacao de todos os workers em um unico ponto. Em producao, os workers rodam como processos separados para isolamento e escalabilidade independente. Em desenvolvimento, todos os workers sao iniciados juntos para facilitar testes. O modulo tambem configura graceful shutdown para garantir que eventos em processamento nao sejam perdidos durante reinicializacoes.

workers/index.ts

```

// workers/index.ts — Orquestrador de Workers
import captureWorker from './captureWorker';
import validateWorker from './validateWorker';
import enrichWorker from './enrichWorker';
import classifyWorker from './classifyWorker';
import actWorker from './actWorker';
const workers = [
  { name: 'capture', worker: captureWorker },
  { name: 'validate', worker: validateWorker },
  { name: 'enrich', worker: enrichWorker },
  { name: 'classify', worker: classifyWorker },
  { name: 'act', worker: actWorker },
];
console.log('[Workers] Iniciando ZEHLA Brain pipeline workers...');
console.log(`[Workers] ${workers.length} workers configurados`);
// Graceful shutdown
async function gracefulShutdown(signal: string) {
  console.log(`\n[Workers] Recebido ${signal}. Encerrando workers...`);
  await Promise.all(
    workers.map(({ name, worker }) =>
      worker.close().then(() => console.log(`[Workers] ${name} encerrado`))
    )
  );
  console.log('[Workers] Todos os workers encerrados com sucesso');
  process.exit(0);
}
process.on('SIGTERM', () => gracefulShutdown('SIGTERM'));
process.on('SIGINT', () => gracefulShutdown('SIGINT'));
export { captureWorker, validateWorker, enrichWorker, classifyWorker, actWorker };

```

app/api/workers/start/route.ts

```

// app/api/workers/start/route.ts — Endpoint para iniciar workers (dev)
import { NextResponse } from 'next/server';

```

```

export async function POST() {
  // Em producao, use PM2 ou Docker para gerenciar workers
  // Este endpoint e apenas para desenvolvimento
  if (process.env.NODE_ENV === 'production') {
    return NextResponse.json(
      { error: 'Workers devem ser gerenciados via PM2 em producao' },
      { status: 403 }
    );
  }
  // Iniciar workers dinamicamente
  const { default: workerModule } = await import('@workers');
  return NextResponse.json({
    success: true,
    message: 'Workers iniciados (modo desenvolvimento)',
    workers: ['capture', 'validate', 'enrich', 'classify', 'act'],
  });
}

```

12. API de Dashboard — Metricas e Monitoramento

O modulo de dashboard fornece endpoints para visualizacao de metricas em tempo real do ZEHLA Brain. Isso inclui distribuicao de leads por cluster, eventos processados por tipo, historico de transicoes, score medio por campanha e acoes executadas. Esses dados alimentam o painel de inteligencia do SMARTHOTEL e o ZEHLA Radar (mapa neural de leads).

app/api/dashboard/brain-metrics/route.ts

```

// app/api/dashboard/brain-metrics/route.ts
import { NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
export async function GET() {
  try {
    const [
      totalLeads,
      clusterDistribution,
      recentEvents,
      recentActions,
      funnelMetrics,
      topScoringLeads,
    ] = await Promise.all([
      // Total de leads
      prisma.lead.count(),
      // Distribuicao por cluster
      prisma.lead.groupBy({
        by: ['cluster'],
        _count: { id: true },
        _avg: { score: true },
      }),
      // Eventos dos ultimos 7 dias
      prisma.leadEvent.groupBy({
        by: ['eventType'],
        _count: { id: true },

```

```

      where: {
        createdAt: { gte: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000) },
      },
    }),
    // Acoes executadas nos ultimos 7 dias
    prisma.actionLog.groupBy({
      by: ['actionType', 'status'],
      _count: { id: true },
      where: {
        createdAt: { gte: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000) },
      },
    }),
    // Metricas do funil
    prisma.lead.groupBy({
      by: ['funnelStage'],
      _count: { id: true },
    }),
    // Top 10 leads por score
    prisma.lead.findMany({
      orderBy: { score: 'desc' },
      take: 10,
      select: {
        id: true, name: true, email: true,
        score: true, cluster: true,
        pousadaName: true, city: true, state: true,
        lastActivityAt: true,
      },
    }),
  ]);
  // Transicoes de cluster (ultimos 7 dias)
  const transitions = await prisma.lead.findMany({
    where: {
      previousCluster: { not: null },
      updatedAt: { gte: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000) },
    },
    select: {
      previousCluster: true,
      cluster: true,
      updatedAt: true,
    },
    orderBy: { updatedAt: 'desc' },
    take: 50,
  });
  // Score medio geral
  const avgScore = await prisma.lead.aggregate({
    _avg: { score: true },
  });
  return NextResponse.json({
    summary: {
      totalLeads,
      avgScore: Math.round(avgScore._avg.score || 0),
    },
    clusters: clusterDistribution.map(c => ({
      name: c.cluster,
      count: c._count.id,
      avgScore: Math.round(c._avg.score || 0),
    })),
  });

```

```

    })),
    eventsByType: recentEvents.map(e => ({
      type: e.eventType,
      count: e._count.id,
    })),
    actionsByType: recentActions.map(a => ({
      type: a.actionType,
      status: a.status,
      count: a._count.id,
    })),
    funnel: funnelMetrics.map(f => ({
      stage: f.funnelStage,
      count: f._count.id,
    })),
    transitions: transitions.map(t => ({
      from: t.previousCluster,
      to: t.cluster,
      date: t.updatedAt,
    })),
    topLeads: topScoringLeads,
    generatedAt: new Date().toISOString(),
  });
} catch (error: any) {
  console.error('[Dashboard] Erro:', error.message);
  return NextResponse.json(
    { error: 'Erro interno do servidor' },
    { status: 500 }
  );
}
}
}

```

13. Client-Side Event Tracking

O modulo client-side fornece funcoes utilitarias para captura de eventos diretamente do frontend Next.js. Isso inclui tracking de page views, cliques em elementos, abertura de emails (via pixel de tracking) e visualizacao de anuncios. As funcoes enviam eventos para o endpoint `/api/events/track` de forma assincrona com fallback para `localStorage` quando a conexao esta indisponivel.

lib/eventTracker.ts

```

// lib/eventTracker.ts — Client-side event tracking
const TRACK_ENDPOINT = '/api/events/track';
interface TrackEvent {
  email: string;
  eventType: string;
  sessionId?: string;
  fingerprint?: string;
  metadata?: Record<string, any>;
}
// Buffer para eventos offline
const eventBuffer: TrackEvent[] = [];
let isFlushing = false;
export function generateFingerprint(): string {
  const { userAgent, language, platform, screen } = navigator;

```

```

const raw = `${userAgent}|${language}|${platform}|${screen.width}x${screen.height}`;
return simpleHash(raw);
}

function simpleHash(str: string): string {
  let hash = 0;
  for (let i = 0; i < str.length; i++) {
    const char = str.charCodeAt(i);
    hash = ((hash << 5) - hash) + char;
    hash |= 0;
  }
  return Math.abs(hash).toString(36);
}

export async function trackEvent(event: TrackEvent): Promise<void> {
  const payload = {
    ...event,
    sessionId: event.sessionId || getSessionId(),
    fingerprint: event.fingerprint || generateFingerprint(),
    metadata: {
      ...event.metadata,
      url: typeof window !== 'undefined' ? window.location.href : '',
      timestamp: new Date().toISOString(),
    },
  };
  try {
    const response = await fetch(TRACK_ENDPOINT, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload),
      keepalive: true, // enviar mesmo se a pagina fechar
    });
    if (!response.ok) {
      eventBuffer.push(payload);
      flushBuffer();
    }
  } catch {
    eventBuffer.push(payload);
    // Tentar flush quando online novamente
    if (typeof window !== 'undefined') {
      window.addEventListener('online', flushBuffer, { once: true });
    }
  }
}

export function trackPageView(email: string): void {
  trackEvent({
    email,
    eventType: 'landing_visit',
    metadata: {
      title: typeof document !== 'undefined' ? document.title : '',
      referrer: typeof document !== 'undefined' ? document.referrer : '',
    },
  });
}

export function trackLinkClick(email: string, url: string): void {
  trackEvent({
    email,
    eventType: 'link_click',
  });
}

```

```

    metadata: { clickedUrl: url },
  });
}
export function trackEmailOpen(email: string, campaignId: string): void {
  trackEvent({
    email,
    eventType: 'email_open',
    metadata: { campaignId },
  });
}
export function trackWhatsAppOpen(email: string): void {
  trackEvent({
    email,
    eventType: 'whatsapp_open',
  });
}
export function trackWhatsAppReply(email: string, messageId: string): void {
  trackEvent({
    email,
    eventType: 'whatsapp_reply',
    metadata: { messageId },
  });
}
function getSessionId(): string {
  if (typeof window === 'undefined') return 'server';
  let sid = sessionStorage.getItem('zehla_session');
  if (!sid) {
    sid = `sess_${Date.now()}_${Math.random().toString(36).slice(2, 10)}`;
    sessionStorage.setItem('zehla_session', sid);
  }
  return sid;
}
async function flushBuffer(): Promise<void> {
  if (isFlushing || eventBuffer.length === 0) return;
  isFlushing = true;
  while (eventBuffer.length > 0) {
    const event = eventBuffer.shift();
    try {
      await fetch(TRACK_ENDPOINT, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(event),
      });
    } catch {
      eventBuffer.unshift(event);
      break;
    }
  }
  isFlushing = false;
}
// React hook para tracking
export function useEventTracker(email: string | undefined) {
  if (typeof window !== 'undefined' && email) {
    // Track page view on mount
    trackPageView(email);
  }
}

```



```

return {
  trackPageView: () => email && trackPageView(email),
  trackLinkClick: (url: string) => email && trackLinkClick(email, url),
  trackEmailOpen: (cid: string) => email && trackEmailOpen(email, cid),
  trackWhatsAppOpen: () => email && trackWhatsAppOpen(email),
};
}

```

14. Configuracao e Deploy

Esta secao detalha todas as variaveis de ambiente necessarias, a configuracao do PM2 para gerenciamento de workers em producao, e um docker-compose completo para deploy da infraestrutura incluindo Next.js, PostgreSQL, Redis e BullMQ Board para monitoramento visual das filas em tempo real.

14.1. Variaveis de Ambiente

Variavel	Descricao	Exemplo
REDIS_URL	URL de conexao Redis	redis://localhost:6379
DATABASE_URL	URL PostgreSQL (Prisma)	postgresql://user:pass@localhost:5432/zehla
WEBHOOK_SECRET	Chave HMAC para webhook verification	whsec_abc123...
EMAIL_API_URL	Endpoint da Delivery Machine	https://email-api.zehla.com/send
EMAIL_API_KEY	API Key do servico de email	sk_live_...
ZEHLA_WHATSAPP_WEBHOOK	Webhook para alertas WhatsApp	https://wa.zehla.com/api/alert
SALES_TEAM_PHONE	Telefone da equipe de vendas	+5511999999999
NODE_ENV	Ambiente de execucao	production

Tabela 3: Variaveis de ambiente obrigatorias

14.2. PM2 Ecosystem Config

```

ecosystem.config.js
// ecosystem.config.js — PM2 para producao
module.exports = {
  apps: [
    {
      name: 'smarthotel-web',
      script: 'node_modules/.bin/next',
      args: 'start',
      instances: 2,
      exec_mode: 'cluster',

```

```

    env: {
      NODE_ENV: 'production',
      PORT: 3000,
    },
  },
  {
    name: 'zehla-worker-capture',
    script: 'workers/captureWorker.ts',
    interpreter: 'node_modules/.bin/ts-node',
    instances: 2,
    env: { NODE_ENV: 'production' },
  },
  {
    name: 'zehla-worker-validate',
    script: 'workers/validateWorker.ts',
    interpreter: 'node_modules/.bin/ts-node',
    instances: 2,
    env: { NODE_ENV: 'production' },
  },
  {
    name: 'zehla-worker-enrich',
    script: 'workers/enrichWorker.ts',
    interpreter: 'node_modules/.bin/ts-node',
    instances: 2,
    env: { NODE_ENV: 'production' },
  },
  {
    name: 'zehla-worker-classify',
    script: 'workers/classifyWorker.ts',
    interpreter: 'node_modules/.bin/ts-node',
    instances: 2,
    env: { NODE_ENV: 'production' },
  },
  {
    name: 'zehla-worker-act',
    script: 'workers/actWorker.ts',
    interpreter: 'node_modules/.bin/ts-node',
    instances: 1,
    env: { NODE_ENV: 'production' },
  },
],
};

```

14.3. Docker Compose

docker-compose.yml

```

# docker-compose.yml
version: '3.8'
services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://zehla:zehla_pass@postgres:5432/zehla_db

```

```

- REDIS_URL=redis://redis:6379
- NODE_ENV=production
depends_on:
- postgres
- redis
restart: unless-stopped
postgres:
image: postgres:16-alpine
environment:
  POSTGRES_USER: zehla
  POSTGRES_PASSWORD: zehla_pass
  POSTGRES_DB: zehla_db
volumes:
- postgres_data:/var/lib/postgresql/data
ports:
- "5432:5432"
restart: unless-stopped
redis:
image: redis:7-alpine
ports:
- "6379:6379"
volumes:
- redis_data:/data
command: redis-server --appendonly yes --maxmemory 512mb --maxmemory-policy allkeys-lru
restart: unless-stopped
bullmq-board:
image: dockersamples/bullmq-board
ports:
- "3001:3000"
environment:
- REDIS_HOST=redis
- REDIS_PORT=6379
depends_on:
- redis
restart: unless-stopped
volumes:
  postgres_data:
  redis_data:

```

14.4. Comandos de Deploy

Terminal — Comandos de Deploy

```

# 1. Instalar dependencias
npm install ioredis bullmq @bull-board/api @bull-board/express

# 2. Rodar migracoes do Prisma
npx prisma migrate deploy
npx prisma generate

# 3. Iniciar infraestrutura
docker-compose up -d postgres redis bullmq-board

# 4. Iniciar aplicacao e workers
pm2 start ecosystem.config.js
pm2 save
pm2 startup

# 5. Verificar status
pm2 status

```

```
pm2 logs zehla-worker-classify --lines 50
# 6. Monitoramento das filas (BullMQ Board)
# Acesse: http://localhost:3001
```

15. Estrutura de Arquivos do Modulo

A estrutura de arquivos segue as convenções do Next.js App Router com organização modular. Os workers ficam na raiz do projeto para fácil acesso via PM2, enquanto os módulos de suporte ficam no diretório lib. Os API routes seguem a estrutura padrão do Next.js em app/api. A separação clara entre camadas permite manutenção independente e testabilidade.

Arquivo	Descrição
prisma/schema.prisma	Modelos LeadEvent, Lead, ActionLog, WebhookLog
lib/redis.ts	Conexão Redis centralizada (ioredis)
lib/queues.ts	Definição das 5 filas BullMQ
lib/prisma.ts	Instância singleton do Prisma Client
lib/eventTracker.ts	Client-side tracking functions + React hook
app/api/events/track/route.ts	Endpoint principal de captura de eventos
app/api/events/webhook/route.ts	Webhook receiver para n8n/Make/Segment
app/api/dashboard/brain-metrics/route.ts	Métricas e KPIs do ZEHLA Brain
app/api/workers/start/route.ts	Endpoint dev para iniciar workers
workers/captureWorker.ts	Worker Estágio 1: Captura + Rate Limiting
workers/validateWorker.ts	Worker Estágio 2: Validação + Deduplicação
workers/enrichWorker.ts	Worker Estágio 3: Enriquecimento contextual
workers/classifyWorker.ts	Worker Estágio 4: Score Engine + Cluster
workers/actWorker.ts	Worker Estágio 5: Action Dispatcher
workers/index.ts	Orquestrador + Graceful Shutdown
ecosystem.config.js	PM2 config para produção
docker-compose.yml	Infra: Next.js + PostgreSQL + Redis + BullMQ Board

Tabela 4: Estrutura completa de arquivos do módulo ZEHLA Brain

16. Fluxo Completo End-to-End

Para consolidar a compreensao de toda a pipeline, esta secao apresenta o fluxo completo de processamento de um evento desde a captura ate a execucao de acoes. O exemplo a seguir mostra como um lead que clica em um link de email (link_click, +15 pontos) e posteriormente responde no WhatsApp (whatsapp_reply, +30 pontos) transicionaria do cluster COLD para WARM e entao para HOT, disparando automaticamente alertas de vendas e sequencias de conversao.

16.1. Exemplo Pratico

Cenario: Maria Silva (maria@pousadasol.com) e proprietaria de uma pousada em Florianopolis. Ela recebeu um email de campanha do ZEHLA e interage com varios canais ao longo de 3 dias.

Timestamp	Evento	Score Impact	Score Total	Cluster	Acao Disparada
Dia 1, 09:00	email_open	+5	5	COLD	-
Dia 1, 09:15	link_click	+15	20	COLD	-
Dia 1, 09:16	landing_visit	+10	30	COLD -> WARM	Email nurture + Remarketing
Dia 2, 14:00	whatsapp_open	+25	55	WARM	-
Dia 2, 14:30	whatsapp_reply	+30	85	WARM -> HOT	Alerta vendas + WhatsApp + Trial
Dia 3, 10:00	trial_started	+50	135	HOT	Email conversao + Bid increase

Tabela 5: Fluxo completo de interacoes da lead Maria Silva

16.2. Log do Pipeline

Abaixo esta o log real que seria gerado durante o processamento dos eventos do exemplo acima. Cada linha mostra o worker responsavel, o evento processado e o resultado da operacao. Este log e essencial para debugging e monitoramento da saude da pipeline em producao.

Pipeline Log — Exemplo Maria Silva

```
[Capture] Evento recebido: email_open (maria@pousadasol.com)
[Capture] Evento recebido: link_click (maria@pousadasol.com)
[Capture] Evento recebido: landing_visit (maria@pousadasol.com)
[Validate] Processando evento: landing_visit para maria@pousadasol.com
[Validate] Novo lead criado: clx7abc123def
[Validate] Job job_001 completado
[Enrich] Enriquecendo evento: evt_001
[Enrich] Geo: Florianopolis, SC, BR
[Enrich] Historico: 3 eventos (30 dias)
[Classify] Lead maria@pousadasol.com: 0 (COLD) -> 30 (WARM) *** TRANSICAO ***
[Act] Processando transicao: COLD->WARM para lead maria@pousadasol.com
[Act] send_nurture_email_welcome: OK
```

```
[Act]      activate_remarketing_campaign: OK
[Act]      Transicao COLD->WARM: 2 OK, 0 FAIL
... 24 horas depois ...
[Capture]  Evento recebido: whatsapp_open (maria@pousadasol.com)
[Capture]  Evento recebido: whatsapp_reply (maria@pousadasol.com)
[Validate] Processando evento: whatsapp_reply para maria@pousadasol.com
[Enrich]   Enriquecendo evento: evt_004
[Classify] Lead maria@pousadasol.com: 55 (WARM) -> 85 (HOT) *** TRANSICAO ***
[Act]      Processando transicao: WARM->HOT para lead maria@pousadasol.com
[Act]      send_sales_alert_hot_lead: OK (canal: whatsapp)
[Act]      send_conversion_email: OK (template: conversion_offer)
[Act]      activate_whatsapp_sequence: OK
[Act]      adjust_ad_bid_increase: OK (multiplier: 1.5)
[Act]      Transicao WARM->HOT: 4 OK, 0 FAIL
```